F C L

The FCL command is an extension of the EXEC command. It operates just like EXEC, but it has some additional features. These features are parameter substitution and controling directives.

DESCRIPTION

The general syntax of the FCL command is:

FCL,<file spec>[,parameter list]

Where <file spec> is the name of a command file. Default extension is .FCL and default drive number is the system drive. [,parameter list] is an optional list of up to nine strings that are used for substitution into the command file. Parameters are delimited by a space or a comma, therefore they may not contain embedded spaces or comma's. The command file is a file of type text. Each line in this file contains either a valid FLEX command, an FCL directive, an empty line or a comment line. Whenever applied to a file that is valid for EXEC, FCL will render the same results, except if the command file contains an ampersand ("&"). Whenever the string "&N" occurs in the command file, the Nth parameter in the parameter-list will be substituted. N is a decimal digit 0-9. Read the paragraph on parameter substitution in macro's in the ASMB manual. The mechanisim is the same. Parameter 0 has special meaning. It represents the string that was entered at <file-spec> in the calling line. (Note, not the entire file specification, just the string that was entered in its place.) If an ampersand should be treated without evoking the substitution mechanism enter two succesive ampersands "&&", they will be read as one ampersand.

EXAMPLE

A file TEST.FCL has been created with the folowing content:

```
DIR &1,&2
LIST $0.FCL
.END
```

It is called as follows:

```
FCL TEST,.TXT,.CMD
```

This will have the same effect as successivly executing the commands:

```
DIR .TXT,.CMD
```

F C L

LIST TEST.FCL

DIRECTIVES

FCL recognises directives in the command file. The directives are: .QUIET .NOQUIET .DO .ENDDO .WHILE .ENDWHILE .IF .ELSE .ENDIF .LET .END. A directive always starts with a "." (decimal point) to easily disciminate it form a FLEX command.

.QUIET

The .QUIET directive suppresses printing of encountered command lines on the terminal. It is assumed by default.

.NOQUIET

The .NOQUIET directive allows printing of encountered FLEX commands on the terminal.

.DO

The .DO command allows a number of commands to be repeated. The syntax is:

.DO <expression>

At this point an explanation on expressions is needed. An expression is a list of terms separated by operators. A term may be a 16 bit unsigned decimal number or a variable. There are 26 variables, represented by the characters A-Z. The valid opereators are "+" and "-". All variables are set to 0 when FCL is called. An example of a valid expression is:

A+B-10

All expressions ara evaluated modulo 2^16. The expression is evaluated and used as the loop counter. Note that variables in the expression are not changed by the .DO directive.

.ENDDO

.ENDDO signals the end of a DO loop. DO loops may be nested. The maximum nesting level is limited only by available memory. Each level takes 4 bytes.

EXAMPLE

```
.DO 5
LIST STARTUP.TXT
.ENDDO
```

The file STARTUP.TXT is listed 5 times.

.WHILE <expression>

.WHILE allows for checking a condition and executing a nummber of lines between a .WHILE and a .ENDWHILE statement. The expression is evaluated and if it is not 0 the lines between .WHILE and .ENDWHILE are executed. .WHILE , .ENDWHILE statements may be nested indefinitly. However each level takes 4 bytes of memory.

EXAMPLE

```
.WHILE A
ECHO Hallo
.ENDWHILE
```

The command "ECHO Hallo" is repeatedly executed as long as the variable A is nonzero.

.IF <expression>

Lines between .IF and .ELSE are executed if the expression evaluates to nonzero. Otherwise the lines between .ELSE and .ENDIF are executed. The use of the .ELSE is optional.

EXAMPLE

```
.IF A
DIR .TXT
.ELSE
DIR .SYS
.ENDIF
```

If A is nonzero DIR .TXT is executed, otherwise DIR .SYS is executed.

.TYPE

The .TYPE directive prints the value of an expression or a literal string on the terminal. The string should be delimited by a pair of double quotes ("). .TYPE tabulates each term. Tabulations are on multiples of colum 8. By entering two comma's in a row, an extra tabulation is performed.

EXAMPLE

```
.TYPE a+b,"Hallo",,c
```

The value of a+b is typed in decimal, followed by a tab and the string "Hallo", followed by two tabs and the value of c in decimal.

.LET sets the value of a variable to some value. The syntax of the .LET directive is:

.LET <variable>=<expression>

The variable is set to the value of the expression.

EXAMPLE

```
.LET b=b+a+2


.END
```

The .END directive stops execution of FCL. Normaly .END is the last line of an .FCL command file. However .END may be in the middle of a command file to terminate a .WHILE, .IF or .DO structure. Also .END is not compulsory at the end of a command file, since an end of file will be treated as a .END directive.

COMMENTS

Any line that starts with an asterisk (*) is treated as comment. It is completly ignored. Blank lines are also ingnored. For clearety of reading lines may be indented. Any leading spaces in a line are skipped.

```
.NOQUIET
       * A DO loop
       .DO 10
                .LET B=5
                * A WHILE in a DO loop
                .WHILE B
                        DIR .TXT
                        .LET B=B-1
                .ENDWHILE
* Two empty lines.


       .ENDDO
.END
```

Upper or lower case is accepted for the directives and the variables A-Z. In FLEX commands upper- or lowercase is treated according to your FLEX conventions.

ERROR HANDLING

Because FCL essentially consists of two parts, the first part reads the input file and puts it into memory, the second part executes it, error handling is at two phases. Some errors can and will be detected in phase one. Unknown directives by *xample are detected at this stage. Also FCL calculates how much user stack is needed for the nested .IF, .WHILE and .DO structures. It reserves 4 bytes for each .IF, .WHILE or .DO encountered. If the total number of .ENDIF's, ENWHILE's or ENDDO's exceeds the total number of .IF's, .WHILE's or .DO's a nesting error results in phase one. A condition where a .DO is terminated by an .ENDIF can not be detected at this stage and may not be detected in phase two, so some care is recommended. In phase two, the execution phase errors like an .IF without an .ENDDIF are detected and reported. If nonexecutable FLEX commands are found FCL relies on FMS to report the error. In any case if an error is detected FCL aborts.

TTYSET PAUSE

During the execution phase of FCL the TTYSET pause is disabled. The old pause is remembered however and restored upon completion or abortion of FCL. Note that called commands that affect the TTYSET pause byte have no effect beyond FCL.